QUEEN'S UNIVERSITY

FACULTY OF APPLIED SCIENCE

DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

SOFT 437 by Dr. Ying Zou

QUIZ 1

Student Name:

Student ID:

Signature:

Question 1 (8 Marks)	
Question 2 (6 Marks)	
Question 3 (12 Marks)	
Question 4 (28 Marks)	
Total Mark (54 Marks)	

Problem 1 -- What is Performance? **(8 marks)**

1) Describe the concepts of ***response time*** and ***throughput*** and give an example of each **(4 marks)**

2) Describe the two important dimensions to software performance timeliness: ***responsiveness*** and ***scalability*** **(4 marks)**

Answer:
1) Response time is the time required to respond to a request
For example, time required for an ATM withdraw transaction.
Throughput is the number of requests that can be processed in some specified time interval.
An ATM network may be required to process 100,000 transactions per hour.

2) Responsiveness is the ability of a system to meet its objectives for response time or throughput
Scalability is the ability of a system to continue to meet its response time or throughput objectives as the demand for its functionality increases.

Problems 2 – What is Software Performance Engineering (SPE)? (6 marks)

A subway pass card is a type of smart card that is used to pay for the public transportation on a subway system. The card can be purchased and loaded with a certain amount of money, and then used to pay for rides on the subway system. A central database is used to maintain the information for each smart card, including the entry point and exit point for each ride, the balance, and the amount of fare deduced each ride. When the card is swiped at the fare gate in the entry point to a subway station, the payment status of the last use is checked by connecting to the central database that manage the balances on the subway pass cards for the city. If a user exits a subway station without pay last time, the user cannot use the card to enter the subway again. The amount of the fare is calculated and deducted when the card is swiped at the fare gate at the exit point. The smart card can be used until the balance is zero.

1) Is performance engineering necessary for this kind of software – why/ why not? **(3 marks)**
2) What kind of performance problems may occur in the smart card application. **(3 marks)**

Answers:

1) The SPE is necessary for this kind of software as the software interacts with the customers, the response time affects the clients' perception of the software. The performance of this software directly affects every customer's activity of the day. Any delay may make the customers miss the trains and therefore cannot catch the schedule appointments. The slow performance of the software would have a large negative impact on the large user base.
2) The performance problems could be the following:
   - Card readers in the fare gate might not have performance issues as it is used by a single user at any time.
   - Databases are centralized and it creates a bottleneck in the processing of the data from the card readers, such as database information retrieval and database information updates.

Problem 3 – Software Architecture Styles **(12 Marks)**

Traditional Enterprise Systems use a centralized server to store and process data. However, such a centralized system is not suitable to process huge volumes of scalable data and creates a bottleneck while processing multiple files simultaneously. Google solved this bottleneck issue using an algorithm called *MapReduce* which consists of *Map phase* and *Reduce phase*. The *Map phase* divides a task into multiple small parts and assigns them to many processors in different locations. Later, the *Reduce phase* collects the results processed in different processors together at one place and integrates them to form the result dataset.

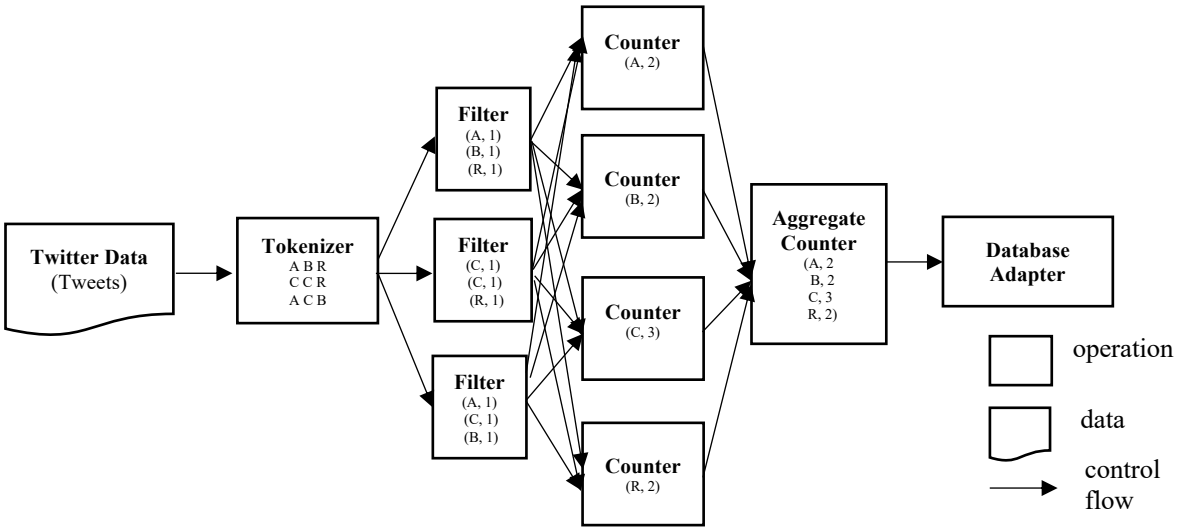
We can use a real-world example to understand the *MapReduce* algorithm. For example, Twitter receives around 500 million tweets per day, which is nearly 3000 tweets per second. The following architecture illustrates how Twitter manages its tweets using *MapReduce* to account the frequency of words appearing the tweets. The functionality of the major components is described as follows:

Tokenizer: Tokenizes the tweets into maps of tokens. For example, sentences are broken down into three tokens, i.e., (A B R), (C C R), (A C B). Each letter represents a token, i.e., a word.

Filter: Filters unwanted words from the maps of tokens and writes the filtered maps as key-value pairs, e.g., (A, 1), (B, 1) and (C, 1).

Counter: Generates a token counter per word.

Aggregate Counter: Prepares an aggregate of similar counter values from smaller manageable units and store the results to the database by the **Database Adapter**.



1) Describe the concept of software architecture style. (**4 Marks**)
2) Describe the software architecture style used in the example above. Please explain the basic structure, and the advantages/disadvantages. (**8 Marks**)

Answers:

1) An Architectural Style defines a family of systems in terms of a pattern of structural organization. It determines:

- the vocabulary of components and connectors that can be used in instances of that style.
- a set of constraints on how they can be combined. For example, one might constrain: the topology of the descriptions (e.g., no cycles).
- execution semantics (e.g., processes execute in parallel).

2) Pipe and filter architecture style is used in the design of the MapReduce algorithm.

A pipe and filter architecture style is suitable for applications that require a defined series of independent computations to be performed on ordered data. A component reads streams of data on its inputs and produces streams of data on its outputs.

- Components: called filters, apply local transformations to their input streams and often do their computing incrementally so that output begins before all input is consumed. In the above architecture, components include Tokenizer, filter, counter, aggregate counter, and database adapter.
- Connectors: called pipes, serve as conduits for the streams, transmitting outputs of one filter to inputs of another. Connectors in the above example transmits each sentence from tokenier to a filter. The filtered words are sent to all the filters to get the count of each word. The counts of each word are sent from a filter to the aggregated counter.

Advantages:

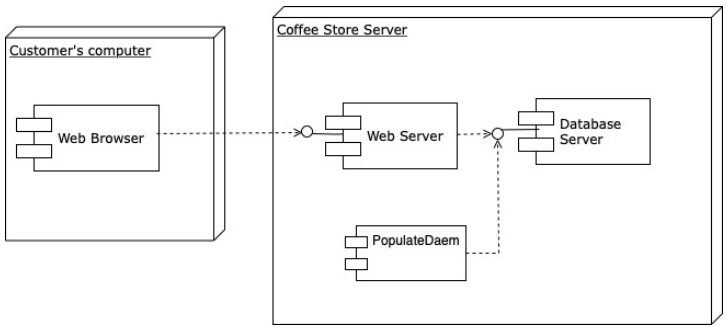
- Systems can be easily maintained and enhanced, since new filters can be added to existing systems and old filters can be replaced by improved ones.
- They permit certain kinds of specialized analysis, such as throughput and deadlock analysis.
- The naturally support concurrent execution.
- Systems can be easily maintained and enhanced, since new filters can be added to existing systems and old filters can be replaced by improved ones.
- They permit certain kinds of specialized analysis, such as throughput and deadlock analysis.
- The naturally support concurrent execution.

Disadvantages:

- Not good for handling interactive systems, because of their transformational character.
- Excessive parsing and unparsing leads to loss of performance and increased complexity in writing the filters themselves.

Problem 4 – Software Execution Models (**28 Marks**)

A local coffee shop offers a reward program. The reward program gives merit points to purchasing products from the coffee store. A customer can order the food using an on-line application. Points are added to a customer's profile if a customer makes a purchase. When the points are accumulated to a certain amount, the customer can receive a free cup of coffee at any size. Once the reward is redeemed, the accumulated points are reset to zero. A reward point record consists of a text string representing the product purchased/redeemed and a positive or negative integer representing the reward points associated with the transaction. The reward program is developed as a Web application which consists of several components displayed in the UML deployment diagram as shown below. Each customer's merit points are stored in the database.



The coffee shop could become very busy during the peak hours. To maintain the system performance, the merit point record for each customer is first written to the memory on the server when the customer makes a purchase. Then a file is created periodically to collect all the merit point records from the memory and store the records in a special directory on the server. The memory is then cleaned out.

Populating database process:

A ***Populate Daemon*** is a special process that periodically checks for the file of storing merit point records. The process sleeps for five minutes, then wakes up and checks to see if the file has appeared in the special directory. If the process doesn't find the file, it goes back to sleep for another ten minutes.

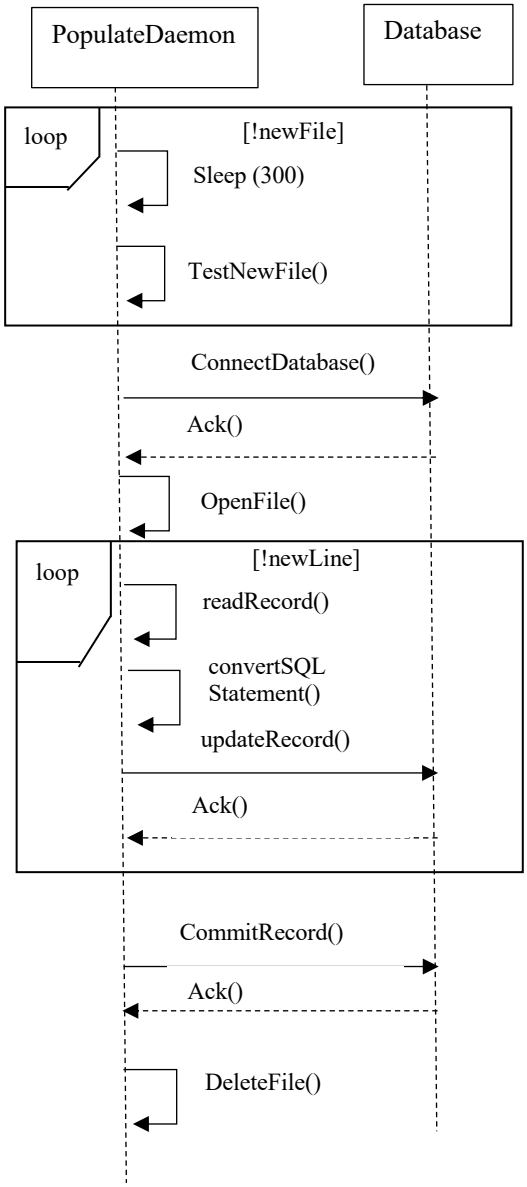
Once the process finds the file, it sends a command to the database server to create a connection to use the database. We can assume that the file typically contains the merit points records for 200 customers. Each customer has an average of 5 records in the file. The ***PopulateDaemon*** opens the file and converts each record in the file into ***SQL update statements*** which are sent to the database server to update the reward points for the customers in the database. When the file has been completely read and the records have been updated in the database, a commit statement is sent to the database server, and the file is deleted. The process then starts checking for a new file again.

(1) Draw a Sequence diagram for the populate database process. (**8 Marks**)
(2) Draw a Software Execution Model diagram using your sequence diagram for the populate database. (**8 Marks**)

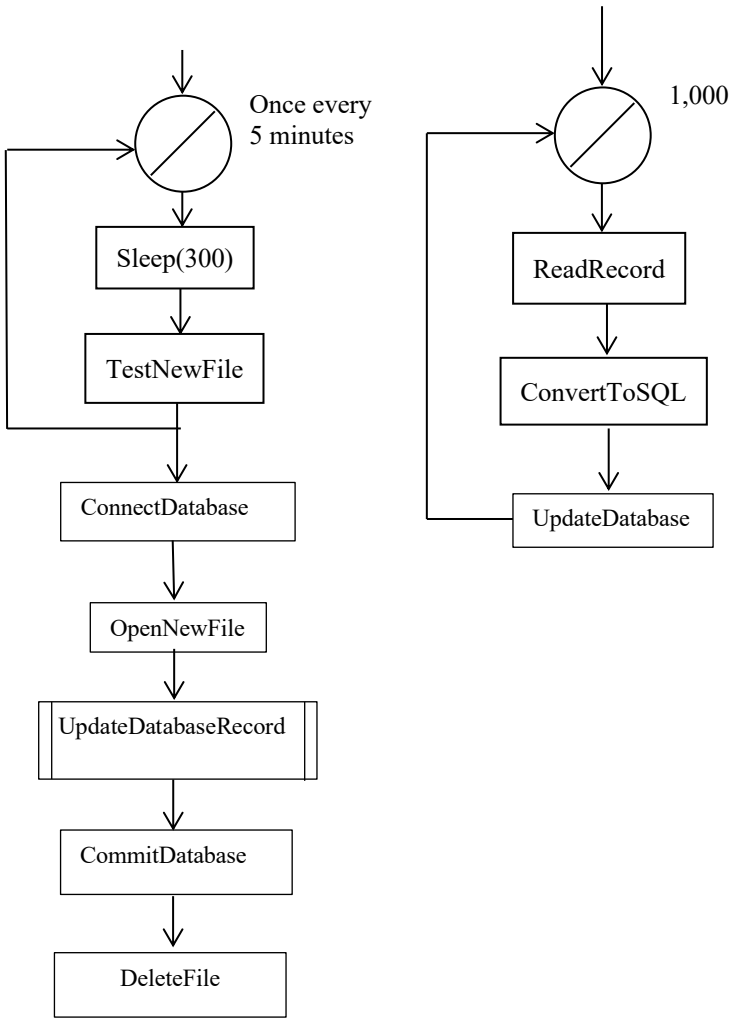
(3) Next to each node in the software execution model diagram, estimate software resources: ***DB***, ***File I/O***, ***Delay*** and ***Msg.*** (**8 Marks**)
(***DB*** refers to the number of database accesses. ***File I/O*** accounts the number of times accessing a file. ***Delay*** is the estimate of the time interval (e.g., in seconds) that an object is paused. ***Msg*** counts the number of messages sent over the network.)

(4) Calculate the ***total software requirement*** for each type of software resources.(**4 Marks**)

Answers: (1)



(2)



(3)
For the subgraph

Software resources	ReadRecord	ConvertToSQL	Update Database
DB	0	0	2
MSG	0	0	0
FileI/O	1	0	0
Delay	0	0	0

Software resources	Sleep	TestNewFile	Connect Database	OpenNewFile	Update DatabaseRecord	Commit Dtabase	Deletefile
DB	0	0	1	0	2,000	2	0
MSG	0	0	0	0	0	0	0
FileI/O	0	1	0	1	1,000	0	1
Delay	300	0	0	0	0	0	0

(4)
Total Software Requirements

DB	2,003
MSG	0
FileI/O	1,003
Delay	300